Lab 11 — Debugging and Testing

Q1. Is there a fault?

Yes. The for-loop begins at i = 1 instead of i = 0. Therefore, attendance_records[0] is never checked. The correct loop is:

for (int i = 0; i < attendance_records.size(); ++i)

Because the first record is skipped, the reported absence count can be one lower than the true count.

Q2. Test that does not execute the fault

Impossible for a valid attendance record of 10 entries. Every valid call enters the loop starting at i = 1, so the faulty loop initialization is executed. There is no valid 10-value attendance record that avoids this statement. Expected and actual outputs for such a test case are not applicable.

Q3. Executes fault, but no error state

Attendance: 1 1 1 1 1 1 1 1 1 1
Expected absence count: 0; actual absence count: 0.
Expected output: PASS; actual output: PASS.

The loop starts at index 1, but index 0 is present (1), so skipping it does not change the absence count.

Q4. Error state without failure

Attendance: 0 0 1 1 1 1 1 1 1 1
Expected absence count: 2; actual absence count: 1.
Expected output: PASS; actual output: PASS.

Skipping index 0 creates an incorrect internal absence count (error state). Both counts remain below 3, so the external result is still correct (no failure).

Q5. Failure

Attendance: 0 0 0 1 1 1 1 1 1 1
Expected absence count: 3; actual absence count: 2.
Expected output: FAIL; actual output: PASS.

The missed absence at index 0 changes the final decision. The program incorrectly reports PASS when the student should FAIL.

Compilation and testing

Save the accompanying source file as lab11.cpp. Compile and run:

```
g++ -std=c++17 -Wall -Wextra -o lab11 lab11.cpp
./lab11
```

The program prints the expected and actual absence counts and results for Q3, Q4, and Q5. It preserves the faulty function so the differences can be demonstrated.